

SetConCA Temporary Project Report

****Architecture, completed work, findings, current position, and next steps**** Date: 2026-05-08 Temporary report artifact.

Executive Summary

SetConCA builds semantic sets from constrained paraphrases, extracts hidden-state activation banks from multiple model families, trains sparse set-level concept codes, then tests whether concept spaces can be bridged across models.

The most important interpretation change so far is that raw TopK overlap is not enough. An earlier internal run reported about 0.695 raw TopK overlap for Gemma 4B -> Llama 8B against a simple 0.25 chance reference. In the current analysis, we found that high raw overlap can survive shuffled target anchors, so the main bridge metric is now real-minus-shuffled TopK.

The strongest current scientific result is that SetConCA beats a pointwise TopK baseline under the shuffled-controlled metric. SetConCA has lower raw overlap than pointwise TopK, but much more anchor-specific signal after subtracting shuffled overlap. This supports the set-level training objective.

Behavioral steering is still early. The concept-code steering proxy is useful, and the first news-prompt causal steering probes show positive keyword gains for a few candidates, but this is not final proof of robust causal control.

Reader Guide: Key Terms

This section explains the words and metrics used in the rest of the report. It is written for a reader who has not followed the project.

Term	Meaning in this project	Why it matters
Original sentence	One source sentence from AG News.	This is the anchor meaning we want all rewrites to preserve.
Rewrite / paraphrase	A model-generated version of the original sentence, with copied keywords banned and word count constrained.	Rewrites give multiple surface forms of the same underlying meaning.
Semantic set	One original sentence plus its accepted rewrites.	SetConCA learns from a group of sentences that should share meaning but differ in wording.
View	One member sampled from a semantic set.	In training, each view becomes one hidden-state vector.
Set size, S	The number of views used from each semantic set during training/evaluation. For example, S=16 means 16 paraphrase views per original.	Larger S gives the model more evidence about what is shared across paraphrases.
Hidden state	A vector taken from inside a language model at a chosen layer.	This is the representation SetConCA learns from.
Activation bank	Saved tensor of hidden states with shape [number of sets, number of views, hidden dimension].	It is the training data for SetConCA after running the language models.
Concept dimension	One coordinate in the learned SetConCA concept space.	We hope some dimensions correspond to reusable semantic factors.

Term	Meaning in this project	Why it matters
TopK	Keep only the k strongest concept dimensions and set the rest to zero. Current standard: C=128 total dimensions, k=32 active dimensions.	This forces a sparse code, making overlap and concept inspection possible.
Bridge	A mapping from one model's concept space to another model's concept space.	This tests whether learned concepts line up across models.
Raw TopK overlap	The fraction of active target concept slots recovered by the bridged source code.	This was the old main metric, but it can be inflated.
Shuffled TopK overlap	The same overlap calculation after pairing source examples with the wrong target examples.	This measures how much overlap happens for boring/global reasons rather than correct semantic matching.
Real - shuffled	Raw TopK overlap minus shuffled TopK overlap.	This is the current main controlled signal. Bigger is better.

The simplest mental model is this: if the bridge is meaningful, it should work better for the correct sentence pair than for a wrong shuffled pair. Raw TopK asks "how much overlap is there?" Real-minus-shuffled asks "how much of that overlap depends on the correct semantic match?"

Architecture Overview

The architecture has two meanings in this project: the end-to-end research pipeline and the neural SetConCA module. The pipeline creates semantic sets, turns them into activation banks, trains sparse concept codes, fits bridges, then inspects and steers candidate concepts.

SetConCA workflow: from paraphrases to causal steering

A simple stage-by-stage view. Each step uses the output from the step above it.

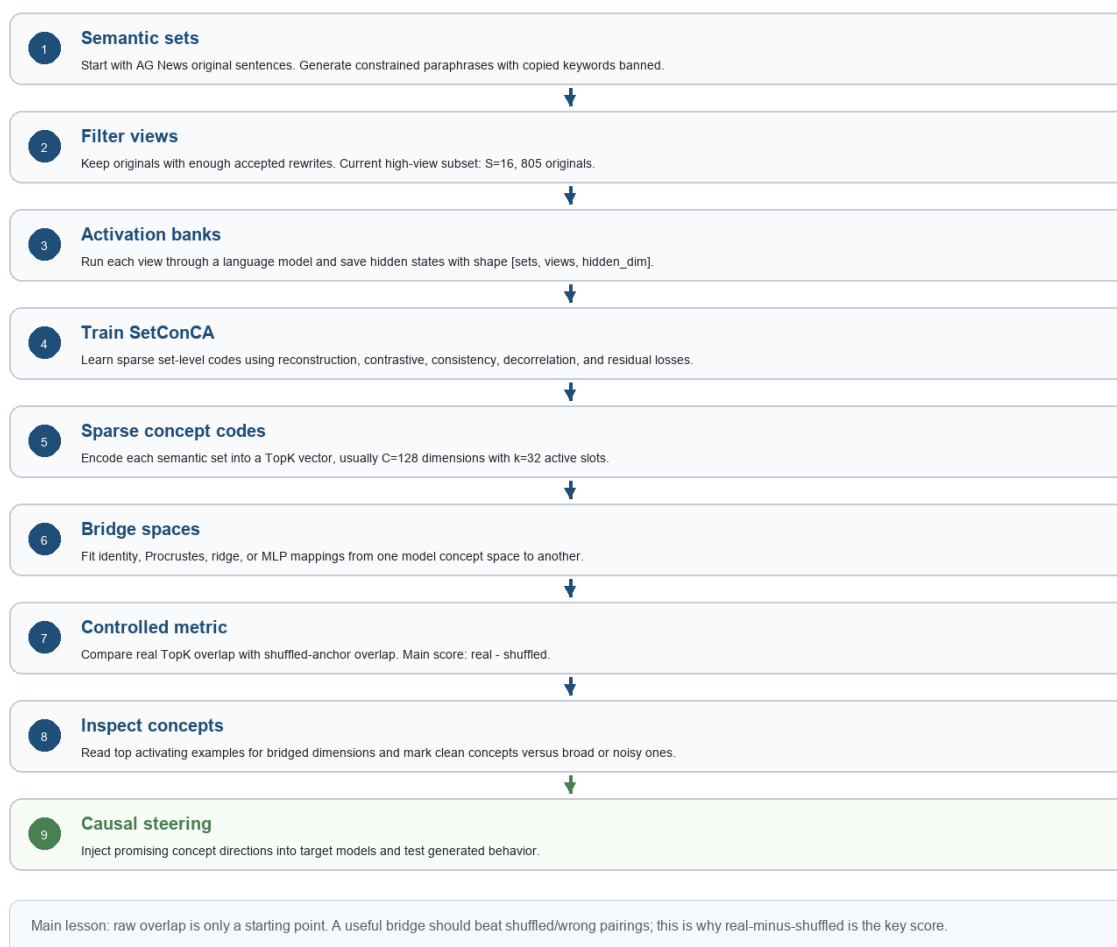


Figure 1. Full SetConCA research pipeline.

SetConCA Model

Input is a set of S paraphrase views for the same original sentence. Each view is represented by a hidden state vector extracted from a target model and layer. A shared linear encoder maps every view into a per-view concept-evidence vector.

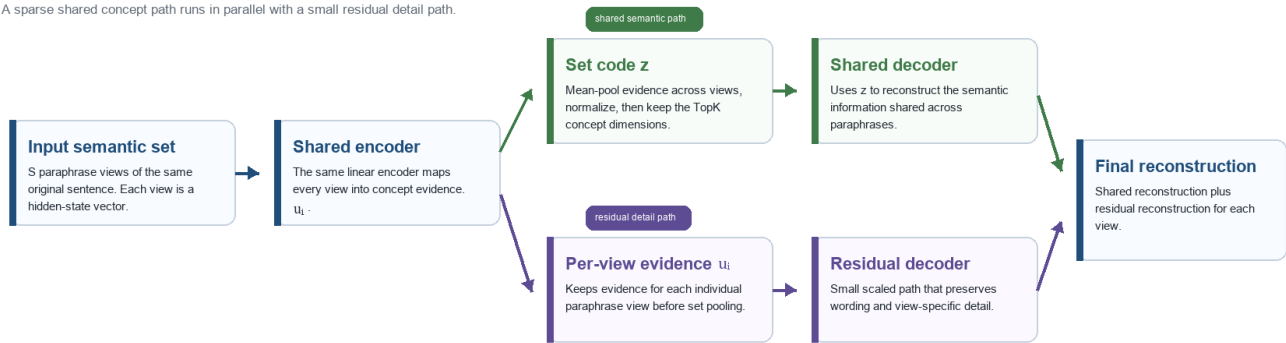
In plain terms, the model does not look at one sentence at a time and ask "how do I reconstruct this exact wording?" Instead, it looks at several rewrites of the same sentence and asks "what information is stable across all of these versions?" That stable information is what we want the sparse concept code to capture.

The model averages the per-view evidence, applies LayerNorm, then keeps only the strongest concept dimensions. In the current standard setup, the concept dimension is 128 and TopK keeps 32 active dimensions. TopK can use absolute value, preserving signed concept evidence.

The decoder is split into a shared decoder and a residual decoder. The shared decoder reconstructs from the sparse set code. The residual decoder reconstructs view-specific detail from each view's evidence vector and is scaled so it cannot absorb the whole task. The intended separation is: the sparse set code carries shared semantic content; residuals carry surface/view detail.

SetConCA model architecture

A sparse shared concept path runs in parallel with a small residual detail path.



Core equations: $u_i = W_e f(x_i) + b_e$ | $z = \text{TopK}(R(\bar{u}_i))$ | $f(x_i) = W_d z + \text{scale} \cdot W_r u_i + b$

Training losses

Shared reconstruction, full reconstruction, contrastive split-view InfoNCE, support consistency, decorrelation, and residual energy. These keep z useful, stable across paraphrase subsets, and protected from residual-path takeover.

Design idea: z should store shared semantic structure. The residual path handles surface variation, so the sparse concept code remains stable and bridgeable.

Figure 2. SetConCA model structure.

Mathematical Formulation

This section connects the original Set-ConCA math to the current implementation. The starting point is Sparse ConCA: a model hidden representation is mapped into a concept-oriented latent code, where each coordinate acts like evidence for one concept.

SetConCA mathematical summary

The set code should capture what remains stable across paraphrase views.

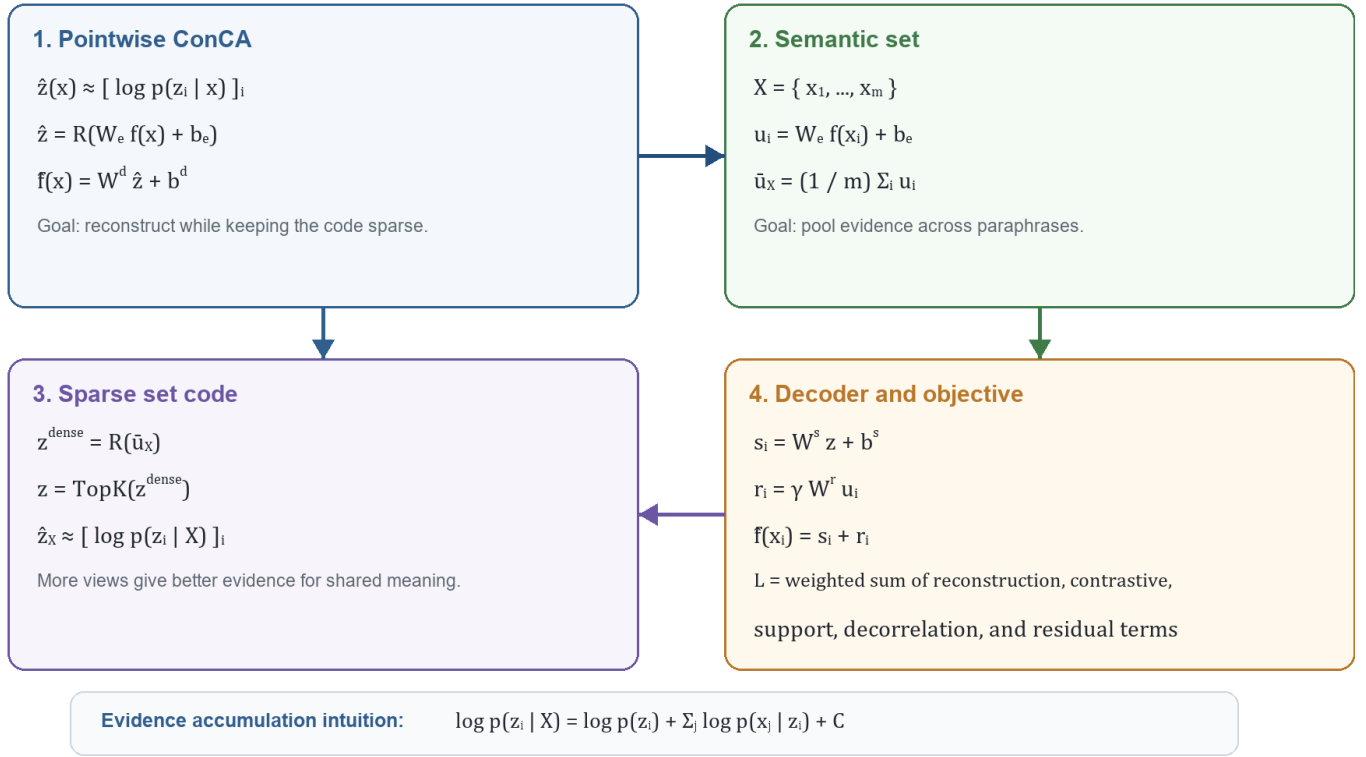


Figure M1. SetConCA mathematical summary.

The original pointwise version takes one sentence representation at a time. It encodes the hidden state into a sparse concept code, then decodes that code back toward the original hidden state. The reconstruction term prevents the code from becoming empty. The sparsity term prevents the code from using every concept coordinate at once.

SetConCA changes the unit of learning from one sentence to a semantic set. In our experiments, a semantic set is one original AG News sentence plus several accepted paraphrase views. Each view is encoded independently into per-view concept evidence. Then the evidence is pooled across views with a permutation-invariant operation. Our practical choice is mean pooling, followed by LayerNorm and TopK sparsification.

The probability intuition is: if several paraphrases express the same underlying concept, then evidence for that concept should accumulate across the set. Mean pooling gives a normalized estimate of this accumulated evidence. This is why set size matters. More paraphrase views should make it easier to identify what survives across rewrites and harder for wording-specific details to dominate.

The decoder has two paths. The shared decoder reconstructs from the sparse set code, so it pressures the code to carry common semantic structure. The residual decoder reconstructs from per-view evidence, but with controlled capacity, so it has a place for wording and view-specific detail without stealing the whole reconstruction task.

The current total objective is the original reconstruction-plus-sparsity idea adapted for sets, plus practical terms that make the learned concepts more stable for cross-model bridge tests: full reconstruction, contrastive split-view alignment, support consistency, decorrelation, and residual energy control.

Training Objective

The current loss combines shared reconstruction, full reconstruction, contrastive split-view InfoNCE, hard-negative margin, support consistency, off-diagonal decorrelation, and residual energy control.

The contrastive term compares codes from two subsets of the same semantic set, encouraging paraphrases of the same sentence to map near each other. The support-consistency term encourages the active sparse support to remain stable across subsets. The decorrelation term discourages every concept dimension from moving together. The residual-energy term keeps the residual path from taking over.

The training objective is trying to balance two pressures. First, the code must keep enough information to reconstruct hidden states. Second, the code should not simply memorize wording or model-specific noise. The set-level and contrastive terms push the code toward the meaning shared by paraphrases, while the residual path gives the model a controlled place to put view-specific detail.

The loss terms are not equally important for the scientific goal. Reconstruction makes the code nontrivial, but the bridge claim depends more on whether the sparse set code is stable, shared, and alignable across models. The figure below gives the clean mathematical summary; the table after it explains the same losses in plain language.

Training objective: what each loss is doing

The losses work together to make the shared code useful, stable across paraphrases, separated into concepts, and protected from residual takeover.

Total objective

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{shared}} + \lambda_2 \mathcal{L}_{\text{full}} + \lambda_3 \mathcal{L}_{\text{contrast}} + \lambda_4 \mathcal{L}_{\text{support}} + \lambda_5 \mathcal{L}_{\text{decor}} + \lambda_6 \mathcal{L}_{\text{residual}}$$

The weights decide how strongly each training pressure contributes.

Shared reconstruction

Make the sparse set code carry common meaning.

High priority

Contrastive alignment

Pull two paraphrase subsets of the same sentence together.

Very high priority

Support consistency

Keep the same TopK concept slots active across subsets.

High priority

Full reconstruction

Check that shared plus residual paths still reconstruct each view.

Medium priority

Residual control

Stop the residual path from doing all the work.

High priority

Decorrelation

Discourage all concept dimensions from moving together.

Medium priority

Takeaway: the bridge claim depends most on shared reconstruction, contrastive alignment, support consistency, and residual control.

Figure M2. Training objective and loss roles.

Loss term	Plain-language job	Why we use it	Importance
Shared reconstruction	Make the sparse set code carry information common to the paraphrase views.	Without this, the shared code could become weak or disconnected from the model representation.	Very high
Full reconstruction	Check that the shared and residual paths together can still reconstruct each individual view.	It keeps the whole autoencoding task grounded, even though it is not the main scientific signal.	Medium

Loss term	Plain-language job	Why we use it	Importance
Contrastive split-view InfoNCE	Split one semantic set into two paraphrase subsets and pull their codes together.	It directly encourages paraphrases of the same original sentence to share a representation.	Very high
Support consistency	Encourage two subsets of the same semantic set to activate the same sparse concept slots.	TopK overlap is meaningful only if the active support is stable across paraphrase samples.	High
Off-diagonal decorrelation	Prevent all learned concept dimensions from moving together.	It encourages different concept dimensions to carry different evidence.	Medium
Residual energy	Keep the residual path controlled.	It prevents the residual decoder from doing all the work and leaving the shared code empty.	High

In short: shared reconstruction, contrastive learning, support consistency, and residual control are the most important for the SetConCA claim. Full reconstruction and decorrelation are supporting terms that make training stable and the representation more useful.

Dataset Work Completed

We generated a constrained paraphrase dataset from AG News. The server run used 2000 original sentences, 10 rewrite models, 4 x NVIDIA A100-SXM4-40GB GPUs, vLLM generation, and four length bands: 5-7, 10-12, 15-17, and 20-22 words.

The strict validation created many attempts because candidates were rejected for banned words, copied terms, or wrong length. This is expected: the goal was not cheap generation, but semantic sets with surface-form pressure.

The banned-word rule matters because an easy paraphrase can copy the most important words from the original sentence. If copies are allowed, two sentences may look similar for trivial surface reasons. By banning likely copied keywords, we force the rewrite models to express the same news meaning with different wording.

Dataset artifact	Rows / sets	Rewrites	Why it matters
sets.jsonl	2000	29183	Full merged generated semantic-set dataset
sets_min8.jsonl	1928	28764	Good for S=8 pilot while preserving nearly all data
sets_min16.jsonl	805	14834	Current experiment subset for S=16, higher-view tests

Semantic-set coverage by minimum rewrite count

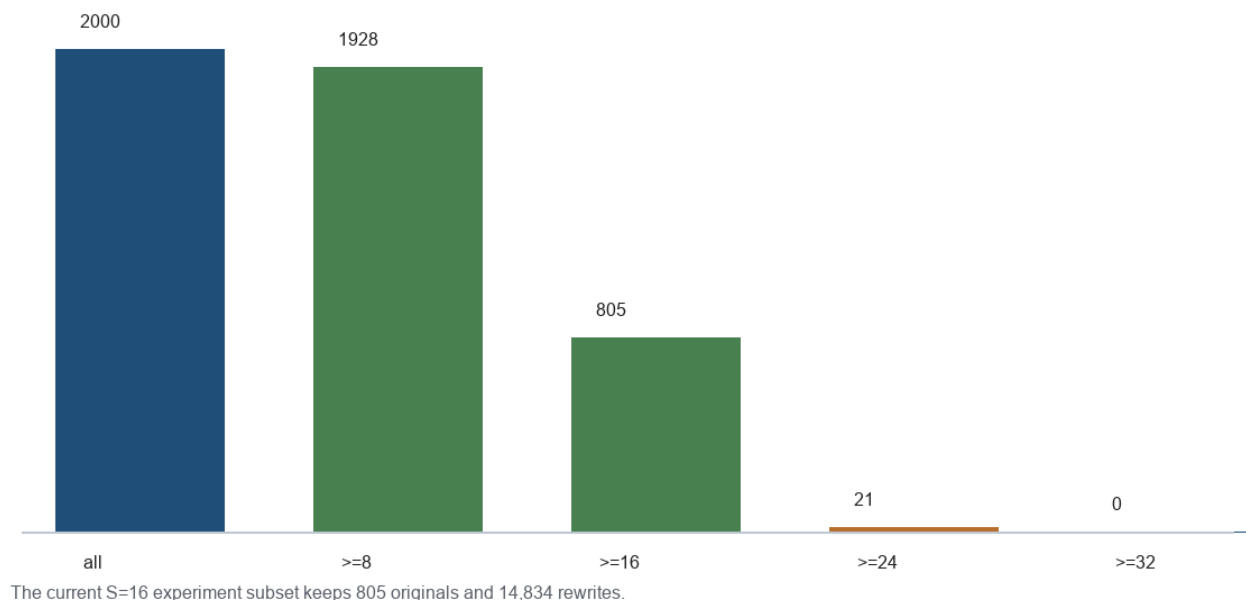


Figure 3. Coverage after filtering by minimum number of rewrites.

Current S=16 Dataset

We created the current experiment subset at `data/generated/server_4gpu_2000/merged/sets_min16.jsonl`. It contains only original sentences with at least 16 accepted rewrites. This is the clean subset for S=16 experiments.

The subset contains 805 original rows, 14,834 rewrites, minimum 16 rewrites, maximum 30 rewrites, and mean 18.4273 rewrites per row. It is smaller than `sets_min8` but gives enough views for stronger set-level tests.

The tradeoff is simple: a higher minimum rewrite count gives better set quality for large-S experiments, but fewer original sentences. The `min8` file keeps almost all data and is good for broad pilots. The `min16` file is smaller but better for testing whether more views improve the learned concept signal.

There is an important data problem right now. The S=16 subset is scientifically useful, but it is not large: only 805 of the 2000 originals survive the at-least-16-rewrites filter. Some later exploratory runs also use a diverse 300-row slice for speed. That makes the current results good for method development, but not yet strong enough for a broad final claim. The project needs either more original sentences, more accepted rewrites per original, or both.

The dataset also has a quality-control limitation. Rewrites are filtered by banned words and word count, but the semantic metric fields are still mostly empty. That means we have not yet automatically checked every rewrite with entailment, contradiction, or embedding-similarity filters. Some generated rewrites preserve meaning well; some are noisy, over-compressed, or accidentally keep strange source artifacts. This matters because SetConCA can only learn clean shared concepts if the paraphrase set really shares the same meaning.

Rewrite count	Rows
16	190
17	142
18	143
19	101

Rewrite count	Rows
20	90
21	51
22	42
23	25
24	14
25	5
26	1
30	1

Activation Extraction And Model Grid

The activation pipeline reads filtered semantic sets, samples a fixed number of views per original, runs a representation model, and saves activation banks as three-dimensional tensors: number of sets by number of views by hidden dimension.

The planned and used model grid covers Llama 3, Gemma 3, and Qwen 3 across small/mid/big sizes and 20%, 60%, and 90% depth layers. This is important because the main question is not one model, but whether concept spaces are bridgeable across families, sizes, and layers.

For each sentence view, we pass the text through a language model and take an internal hidden vector from a specific layer. A 20% depth layer is relatively early, 60% is middle, and 90% is late. Testing multiple depths helps us see where concept-like representations are easiest to align.

There are two different meanings of "model" in the project. Rewrite models are used only to generate paraphrases. Representation models are the models whose hidden states we study. The bridge experiments are about the representation models, not about the rewrite models.

Model family	Role in this project	Sizes used in the grid	What it helps us test
Llama 3	Representation model family from Meta. We extract hidden states and train SetConCA codes on them.	small 1B, mid 3B, big 8B	Whether concepts transfer within one family and from/to a widely used open model family.
Qwen 3	Representation model family from Qwen. We extract hidden states and compare against Llama and Qwen variants.	small 0.6B, mid 4B, big 8B	Whether SetConCA concepts bridge across a different architecture/training family.
Gemma 3	Representation model family from Google. It is included as another independent family.	small/mid/big variants in the family grid	Whether high raw overlap is real semantic transfer or mostly a shuffled-control artifact.
Rewrite models	Generation models used to create paraphrases before activation extraction.	10 generator models in the dataset run	They create surface diversity. They are not the same thing as the representation models being bridged.

The size labels are practical labels, not claims that one model is always better. "Small", "mid", and "big" let us ask whether a concept learned in a smaller model can be mapped into a larger model, or whether the bridge only works when source and target have similar capacity. The layer-depth labels ask a different question: are shared concepts easier to find early, in the

middle, or late inside the model?

Earlier Raw TopK Result Reinterpreted

The earlier internal run reported Gemma 4B -> Llama 8B at about 0.695 TopK overlap, with $k/C = 32/128 = 0.25$ as a simple chance reference. That number measured raw overlap between bridged source TopK support and true target TopK support.

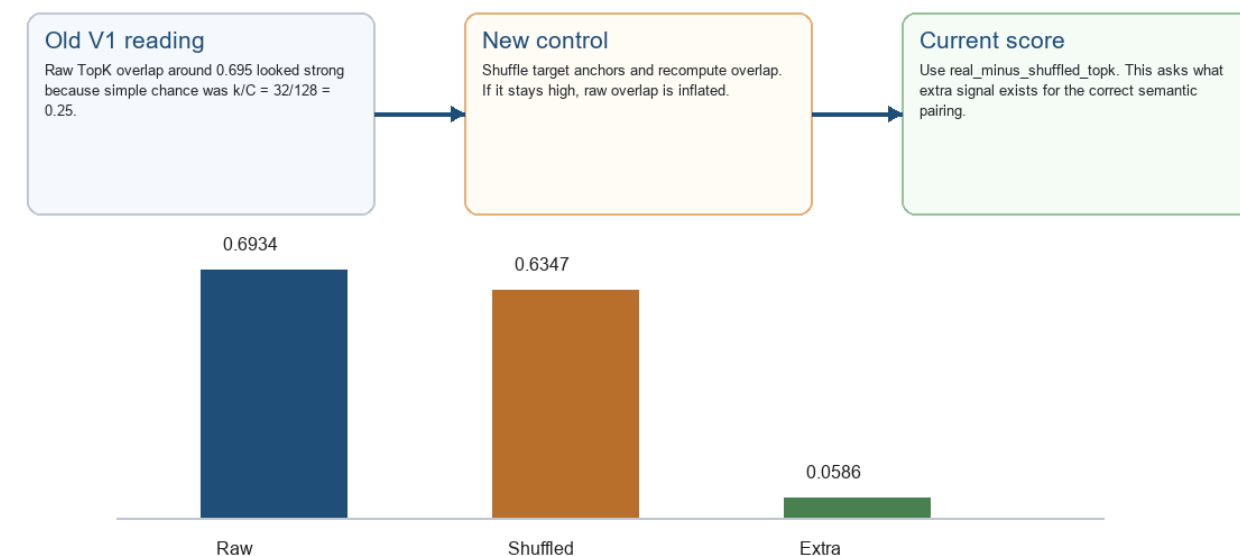
The current diagnostic finding is that raw overlap can be optimistic. If target anchors are shuffled and overlap remains high, then part of the raw score is not anchor-specific semantic transfer. Therefore we use the real-minus-shuffled TopK score as the main first-pass bridge metric.

The overlap calculation is the size of the intersection between the active target concepts and active predicted concepts, divided by k . With $k=32$, if 20 active concept slots match, the overlap is $20/32 = 0.625$.

The shuffled-control calculation repeats the same formula, but compares each source example to the wrong target example. If the shuffled score is high, then the bridge is probably matching common active slots or global code patterns, not the specific meaning of the correct sentence.

The controlled score is raw TopK overlap minus shuffled TopK overlap. For example, if raw overlap is 0.6934 and shuffled overlap is 0.6347, then the controlled score is 0.0587. This means only about 0.0587 of the overlap is extra signal from the correct pairing.

Why the old 69% raw TopK result changed meaning



Example: broad cross-family MLP summary. The raw number resembles V1, but the controlled extra signal is much smaller.

Figure 4. Raw TopK can look strong while the controlled extra signal is small.

Family Bridge Diagnostics

The family reruns showed identity bridges stay near 0.25, which is good: concept axes are not trivially identical. Llama 3 gives the strongest controlled linear signal. Qwen 3 gives a moderate controlled signal. Gemma 3 has extremely high raw overlap, but almost all of it remains under shuffled controls, so it should be interpreted cautiously.

The identity bridge is a useful sanity check. It means "do not learn a mapping; just compare the spaces as they are." Since $k/C = 32/128 = 0.25$, identity near 0.25 says the spaces are not already aligned by accident. If identity were very high, it would be hard to know whether the bridge learned anything.

The bridge types mean:

Bridge	What it does	Why we use it
Identity	No learned mapping.	Chance/sanity baseline.
Procrustes	Learns a rotation/reflection-like linear alignment.	Tests whether spaces are alignable without changing scale too freely.
Ridge	Learns a regularized linear map.	Main practical linear bridge; more flexible than Procrustes.
MLP	Learns a nonlinear mapping.	Useful upper comparison, but less central to the ConCA linear-bridge claim.

Family	Bridge	Raw TopK	Shuffled TopK	Real - shuffled
Qwen 3	Procrustes	0.6410	0.5345	0.1065
Qwen 3	Ridge	0.7266	0.6014	0.1252
Llama 3	Procrustes	0.5107	0.3229	0.1878
Llama 3	Ridge	0.5130	0.3328	0.1802
Gemma 3	Procrustes	0.9520	0.9225	0.0294
Gemma 3	Ridge	0.9646	0.9323	0.0323

SetConCA Versus Pointwise TopK

This was one of the strongest current results. The pointwise TopK baseline trains an encoder on individual views and pools codes only after training. It has high raw overlap, but very high shuffled overlap too. SetConCA has lower raw overlap but much higher real-minus-shuffled overlap.

This supports the central design choice: set-level training is doing more than simple sparse coding plus post-hoc pooling.

This table can be confusing because pointwise TopK has the larger raw score. The important point is that pointwise also scores very high after shuffling. For pointwise Ridge, raw overlap is 0.8693, shuffled overlap is 0.8106, and the controlled signal is only 0.0586. For SetConCA Ridge, raw overlap is lower at 0.6107, but shuffled overlap is also lower at 0.4658, giving a controlled signal of 0.1449.

So SetConCA has lower raw overlap, but more of its overlap depends on the correct source-target pair. In ratio form, 0.1449 divided by 0.0586 is about 2.47. That means SetConCA Ridge has about 2.47 times more shuffled-controlled signal than pointwise Ridge in this summary. That is why the difference between real and shuffled being bigger is better: it means the match is less explainable by generic active slots and more explainable by the correct semantic anchor.

Method	Bridge	Raw TopK	Shuffled TopK	Real - shuffled
Pointwise TopK	Procrustes	0.8251	0.7787	0.0465
Pointwise TopK	Ridge	0.8693	0.8106	0.0586
SetConCA	Procrustes	0.5432	0.4099	0.1333
SetConCA	Ridge	0.6107	0.4658	0.1449

SetConCA Gives The Stronger Controlled Linear Bridge Signal

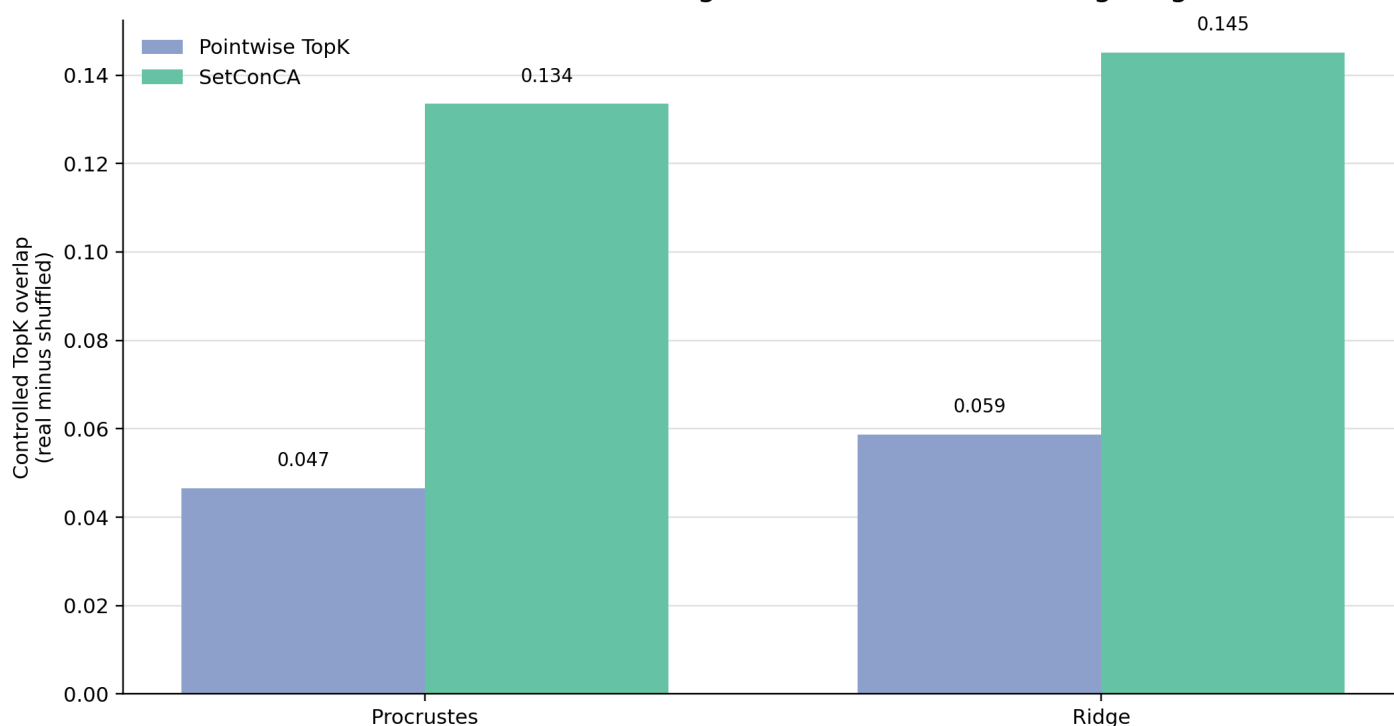


Figure 5. Method/bridge comparison under shuffled-controlled overlap.

Set Size Pattern

SetConCA improves strongly as set size increases from 2 to 16. Pointwise TopK remains much flatter and weaker under the controlled metric. This is exactly the pattern we would hope to see if multiple paraphrase views help isolate shared semantic structure.

Set size S is the number of paraphrase views used together. If $S=2$, the model sees only two versions of the same underlying sentence. If $S=16$, it sees sixteen versions. More views make it easier to separate meaning from wording because the wording changes many times while the meaning should stay stable.

The expected pattern is: larger S gives a better estimate of shared meaning, which should give a stronger controlled bridge signal. That is what we see for SetConCA. Pointwise TopK does not benefit much because it was trained on individual views first; it only pools after training, so the set structure does not shape the code in the same way.

S	SetConCA Procrustes	SetConCA Ridge	Pointwise Procrustes	Pointwise Ridge
2	0.0974	0.1004	0.0458	0.0533
4	0.1138	0.1184	0.0376	0.0449
6	0.1244	0.1343	0.0429	0.0534
8	0.1349	0.1472	0.0453	0.0578
10	0.1426	0.1551	0.0471	0.0612
12	0.1446	0.1618	0.0481	0.0631
14	0.1518	0.1673	0.0506	0.0662
16	0.1568	0.1745	0.0542	0.0694

Larger Semantic Sets Improve SetConCA Transfer

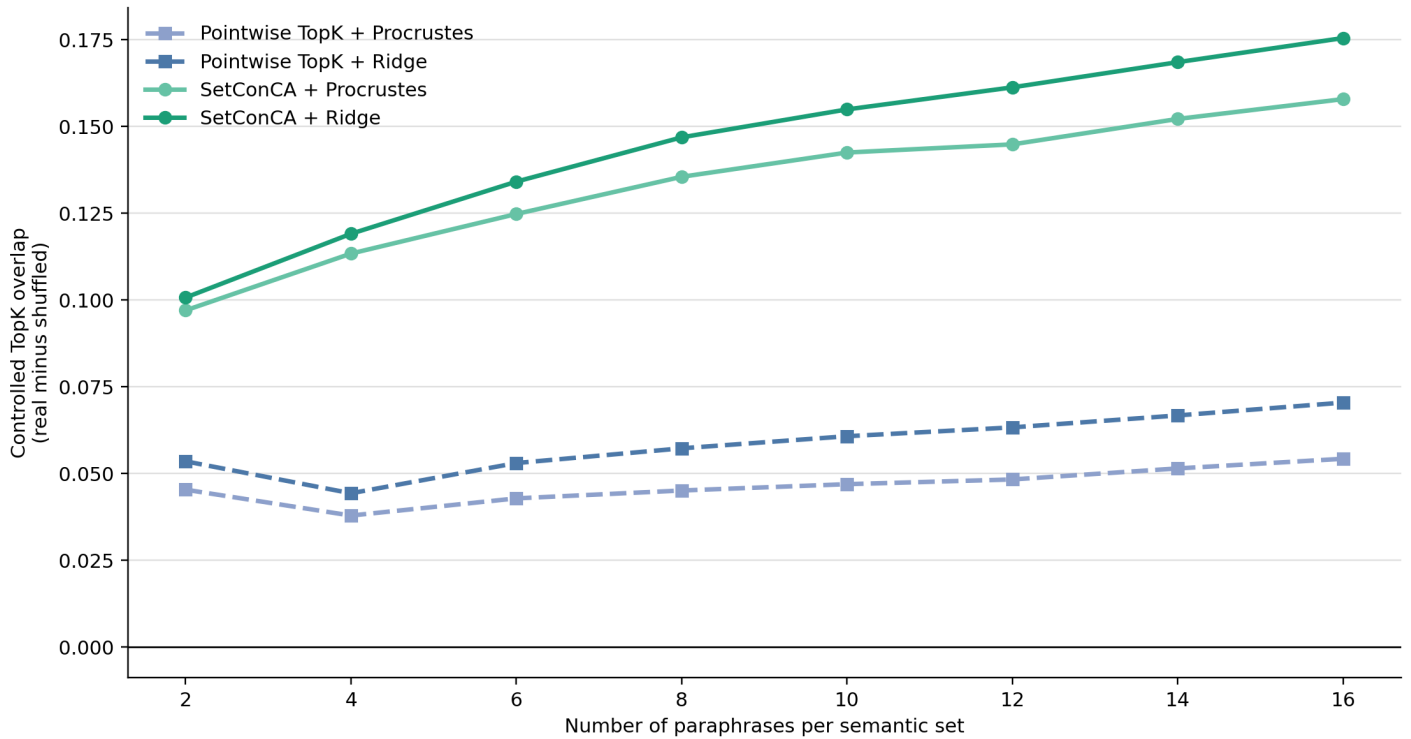


Figure 6. Controlled overlap improves with set size for SetConCA.

Epoch Sweep

We compared 25, 50, and 100 epochs. Longer training did not improve SetConCA controlled bridge signal. SetConCA was best at 25 epochs for both Procrustes and ridge, while pointwise TopK improved with more epochs but still remained below SetConCA after shuffled correction.

This matters because more reconstruction optimization does not automatically produce more transferable concept coordinates.

This is important because "better training loss" and "better concept transfer" are not the same thing. Longer training can improve reconstruction while making the code more specialized to details that do not transfer cleanly. For the current goal, the controlled bridge score matters more than reconstruction alone.

Run	SetConCA Procrustes	SetConCA Ridge	Pointwise Procrustes	Pointwise Ridge
e25	0.1335	0.1452	0.0466	0.0587
e50	0.1303	0.1416	0.0591	0.0738
e100	0.1263	0.1368	0.0732	0.0886

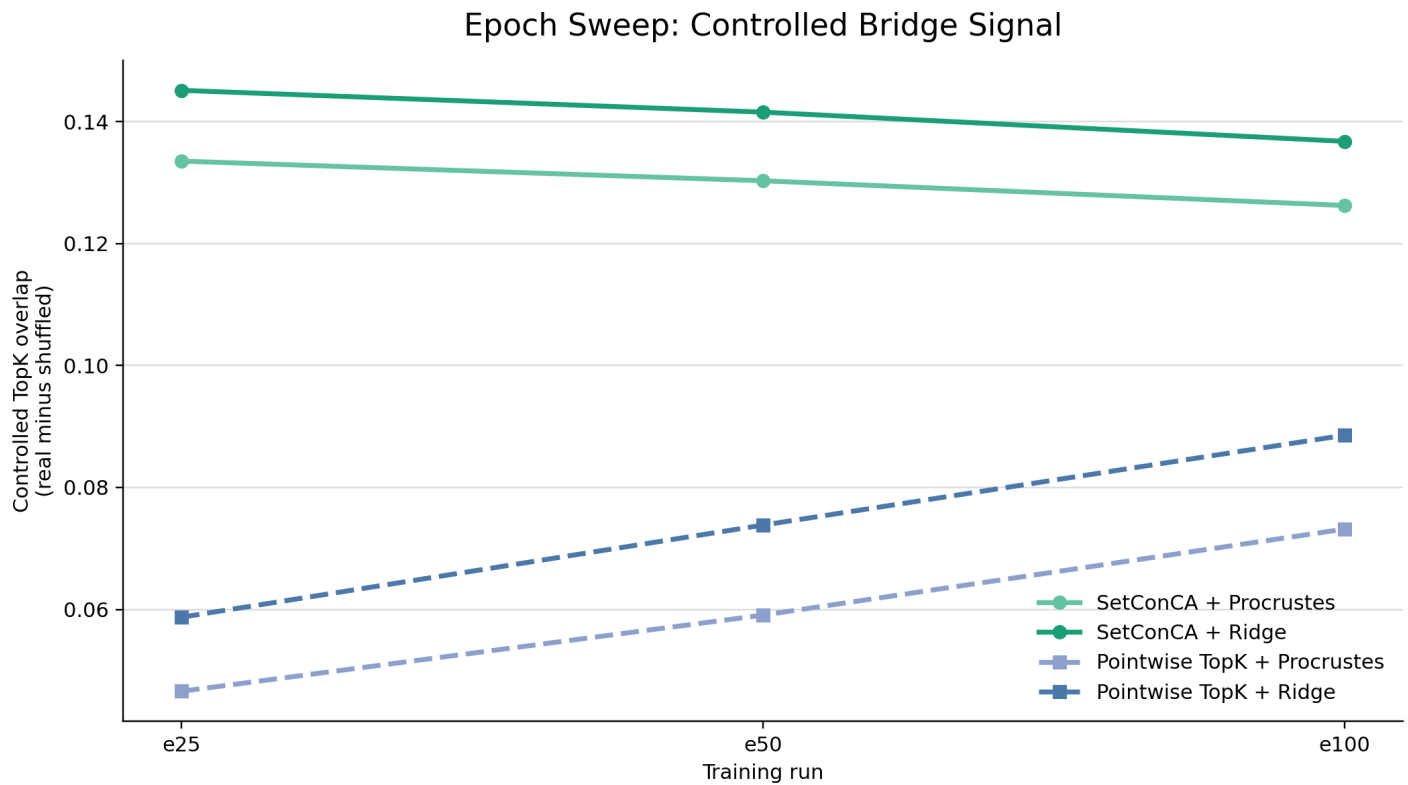


Figure 7. Epoch comparison for controlled bridge signal.

Concept Inspection

After aggregate bridge scores, we moved to inspectable concepts. The inspection script selects strong source/target pairs, fits the bridge, scores individual target concept dimensions on held-out examples, and attaches original text plus rewrite examples.

The inspection score is a ranking aid that combines two controlled signals: alignment above shuffled alignment, and top-example overlap above shuffled top-example overlap. It helps choose candidates worth reading, but it is not proof of monosemanticity.

The reason for concept inspection is that a good aggregate score does not tell us what a concept means. Inspection asks: for one learned concept dimension, which examples activate it most strongly? If the top examples all share a recognizable theme, the dimension may be interpretable. If they are mixed, the dimension may be broad, noisy, or an artifact.

Early candidates include within-family and cross-family pairs, with themes such as market uncertainty, disasters, conflict, and technology/autonomy. Evidence is mixed: some concepts look plausible, some broad, some unclear.

Figure 8 is a ranking chart for candidate bridged concept dimensions. Each row has the form "source model -> target model | cN". The source model is where the concept direction starts. The target model is where we test whether the bridged direction lands. The cN label is the target concept dimension number, for example c126 means target concept dimension 126.

The horizontal bar length is the concept inspection score. This score combines two controlled checks: whether the bridged source dimension aligns with the target dimension better than a shuffled control, and whether the top examples for that concept overlap better than a shuffled control. A higher bar means "worth reading first", not "definitely a clean human concept".

Figure 8 item	Meaning
llama3 mid_3b -> llama3 big_8b	A concept from the 3B Llama space is bridged into the 8B Llama space. This is within-family transfer.

Figure 8 item	Meaning
qwen3 big_8b -> llama3 mid_3b	A concept from Qwen 8B is bridged into Llama 3B. This is cross-family transfer.
qwen3 big_8b -> qwen3 mid_4b	A concept from Qwen 8B is bridged into Qwen 4B. This is within-family but across size.
c126, c37, c92, etc.	The target concept dimension being inspected. These are learned SetConCA coordinates, not manually named topics.
Score around 1.3 to 1.5	Strong under this diagnostic, but still requires manual reading of top examples before we name the concept.

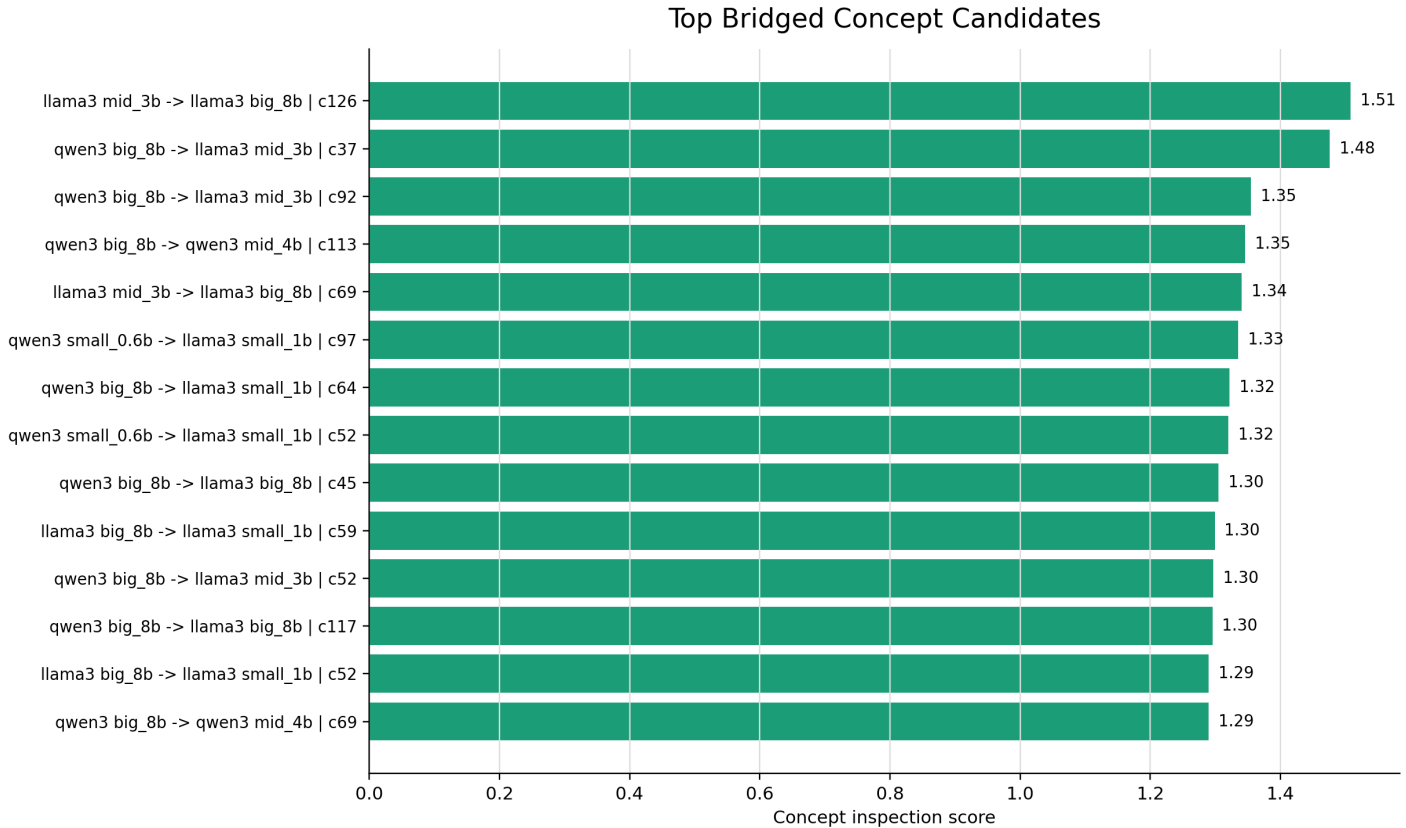


Figure 8. Top bridged concept candidates from inspection.

Causal Steering So Far

The steering proxy tests whether bridged concept directions move target concept codes more than random directions. This is not a full behavioral intervention. It is a concept-code steering proxy.

The intuition is simple. After SetConCA learns a concept code, one concept dimension corresponds to a direction in hidden-state space through the decoder. If the bridge says "this source concept matches that target concept", we can take the target-side direction and add a small amount of it into the target model's hidden state during generation. The alpha value controls how strongly we add the direction. Alpha 0 is the baseline with no steering; larger alpha values are stronger interventions.

This is why the word "causal" must be used carefully. A strong causal steering result would mean that adding the direction reliably changes generated text in the intended semantic direction, while matched control directions do not. What we have right now is a proof of concept: the hook works, the generations can be moved, and several candidates show positive keyword movement. It is not yet proof of robust semantic control.

The latest steering comparison tested active concept directions against an opposite-active condition. The active direction is the direction chosen from the concept's observed activation pattern. The opposite-active condition is a stronger sanity check than simply doing nothing: it asks whether the apparent gain is direction-specific or whether almost any related intervention can increase the keyword score.

The result is encouraging but not clean enough for a final claim. Active steering gives the strongest gains for Google IPO / stock offering and corporate earnings / company performance, each reaching a best keyword gain of 0.50. It also gives smaller positive gains for Windows / software security updates, software / IT products, and stock market / earnings / prices. However, the opposite-active condition also gives positive gains for some finance concepts, including Google IPO / stock offering, stock market / earnings / prices, and corporate earnings / company performance. That means the current result shows that concept-derived interventions can move behavior, but it does not yet prove that the learned direction is uniquely controlling the intended concept.

There are two different steering levels:

Steering level	What it tests	What it does not prove
Concept-code steering proxy	Whether a concept direction moves internal target concept codes more than random directions.	It does not prove the generated text changes in the intended way.
Causal generation steering	Whether injecting a direction during generation changes model output toward the concept.	Early keyword gains do not yet prove robust semantic control.

The current behavioral probe uses keyword score because it is fast and easy to inspect. It is only a first diagnostic. Keyword gain can be misleading: a generation can mention the right word without expressing the right concept, or express the right concept with different words and receive no credit. It can also reward broad topic drift rather than precise semantic steering.

The data problem is now the main limitation. The latest probe still uses only a small number of candidates and only six prompts per candidate. That is enough to check whether the mechanism can affect generation, but it is too small for a stable estimate of success rate. The prompt set is also news-topic-heavy, so finance concepts may look better partly because the prompts and keyword lists match the dataset style. This is useful for proof of concept, but it is not yet a general steering benchmark.

The next steering dataset should include many more held-out prompts per concept, positive and negative steering signs, random directions, opposite-active directions, repeated seeds or deterministic decoding, and a semantic evaluator that can judge meaning rather than only keyword presence. Until that is done, the honest claim is: SetConCA directions can be injected into language-model generation and can produce measurable concept-related shifts in some cases, but the causal steering evidence is preliminary.

Candidate	Target model	Best alpha	Baseline score	Steered score	Gain
Google IPO / stock offering	Llama-3.2-3B	1.0	0.17	0.67	0.50
Corporate earnings / company performance	Llama-3.1-8B	2.0	1.17	1.67	0.50
Windows / software security updates	Llama-3.2-1B	0.5	0.83	1.00	0.17
Software / IT products	Llama-3.2-3B	2.0	0.17	0.33	0.17
Stock market / earnings / prices	Qwen3-4B	2.0	0.33	0.50	0.17

The opposite-active control is important. For some concepts it stays at zero, which supports direction specificity. For other concepts it also produces positive keyword gain, which weakens the causal interpretation. The control therefore does exactly what it should do: it prevents us from over-claiming from a small keyword-based probe.

Figure 9 should be read as a proof-of-concept steering diagnostic. Each small panel is one candidate concept. The x-axis is alpha, the strength of the hidden-state intervention. The y-axis is mean keyword gain over six prompts, relative to the unsteered baseline. The blue line is the active concept direction. The orange line is the opposite-active control. Blue above orange is the pattern we want, because it suggests the intended direction is doing more than a generic intervention. Orange above zero is a warning sign, because it means the control can also increase keyword score.

Causal steering proof-of-concept: active direction vs opposite-active control

Each panel shows mean keyword gain over 6 prompts. Values above zero mean the steered output used more target keywords than baseline.

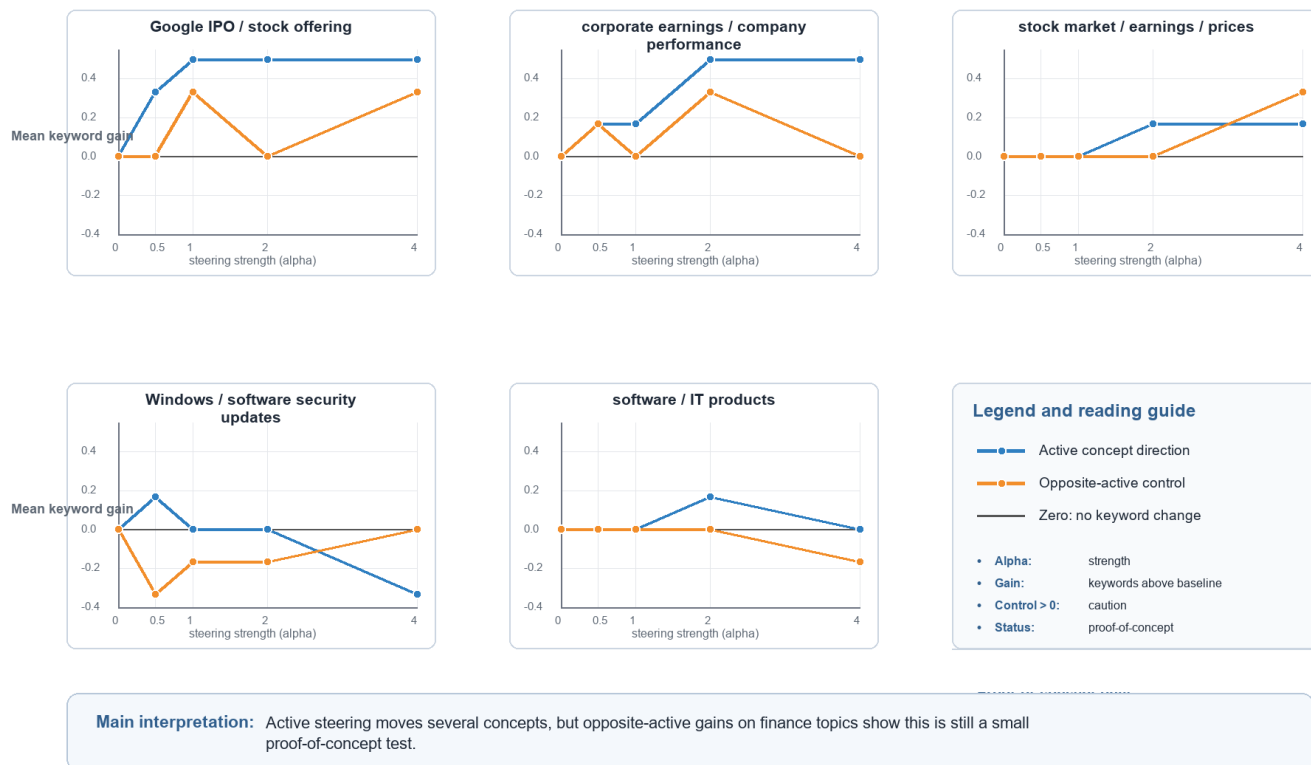


Figure 9. Active steering compared with opposite-active control.

What We Think So Far

The old raw 69% TopK story was too simple. It was not useless, but it was incomplete. The current controlled metric is more honest because it asks whether the correct semantic pairing beats shuffled anchors.

SetConCA currently looks scientifically stronger than pointwise TopK when judged by anchor-specific controlled transfer. This is the most important positive result.

Set size matters. The S=16 subset is worth using even though it is smaller, because the controlled signal improves as more paraphrase views are available.

Gemma needs caution because high raw overlap is mostly explained by shuffled controls. Llama and Qwen, especially qwen3 -> llama3 and llama3 within-family, are more promising.

Causal steering is promising but not done. We have early positive probes, not a final behavioral claim.

Where We Are Right Now

The data pipeline is usable and has produced full, min8, and min16 datasets. The min16 dataset is the active subset for higher-view experiments.

Activation extraction and training infrastructure are in place. The transfer/steering grid can compare SetConCA and pointwise TopK across families, bridges, set sizes, and epochs.

The main metric has matured from raw TopK overlap to shuffled-controlled real-minus-shuffled TopK. This changed the interpretation of earlier results and made the current evidence cleaner.

Concept inspection exists and has produced candidate concepts. First causal steering probes exist and show partial success.

Next Logical Steps

1. Build a more robust dataset before making stronger causal-steering claims. The current steering experiments have too few usable concept sets and too few prompts per concept, so the results are useful as proof of concept but not enough for a stable behavioral conclusion.
2. Broaden the source domains beyond news. AG News gives different news types, but they are still all news-style text. That means concepts may be biased toward headlines, business events, sports reporting, world events, and technology news. A stronger dataset should include broader language domains such as general factual statements, science, health, everyday situations, instructions, product descriptions, biographies, reviews, and possibly synthetic controlled topics.
3. Generate more accepted rewrites per original and more original sets overall. The S=16 filter leaves only 805 sets from the 2000 originals, and some steering work used only a 300-row slice. For robust steering and concept inspection, we need many more high-quality semantic sets that survive the rewrite-count filter.
4. Improve rewrite quality checks. Banned words and word-count constraints are useful, but they are not enough. The next dataset should add semantic filtering: entailment checks, contradiction rejection, embedding similarity, topic consistency, and maybe a small manual audit. The goal is to keep paraphrases that preserve meaning while still changing surface wording.
5. Design the next dataset deliberately, not only by scaling the current generator. We need to decide which domains, sentence lengths, concept types, rewrite models, acceptance thresholds, and validation metrics will best test SetConCA. A good next step is to write a dataset plan that defines the target number of originals, rewrites per original, domain balance, quality filters, and held-out steering prompts.
6. After the broader dataset exists, rerun the main checks: SetConCA versus pointwise TopK, set-size scaling, shuffled-controlled bridges, concept inspection, and causal steering with active, opposite-active, and random controls. This will show whether the current findings survive outside the narrow news setting.
7. Keep the current S=16 and 25-epoch SetConCA setup as the working baseline while building the new dataset. This lets us compare the old news-only result against the next broader dataset without changing too many variables at once.